



YAŞAR UNIVERSITY

COMP 3304:FUNDAMENTALS OF SOFTWARE ENGINEERING

TERM PROJECT PROGRESS REPORT

UNIRIDE

ENGINEERING FACULTY

COMPUTER ENGINEERING PROGRAM

**BORNOVA / İZMİR
MARCH 2026**

TABLE OF CONTENTS

1.1 Project Charter & Vision.....	3
1.2 Personas, Scenarios, User Stories & Feature List.....	5
1.3 Architecture Diagrams.....	6
1.4 UML Diagrams.....	7
1.5 Selected Design Pattern.....	9
1.6 GitHub Repository.....	10
1.7 Jira Project Management.....	11

1.1 Project Charter & Vision

Project Name: UniRide

Product Vision: UniRide is a peer-to-peer car-sharing ecosystem centered around university campuses. It focuses on creating a trust-based community where student car owners can generate extra income while students in need of transportation can access safe and affordable vehicles near their campus.

The Problem: University students face high costs, distant locations, and rigid procedures when renting from traditional car rental companies. Simultaneously, students who own cars cannot easily monetize their vehicles during days they are not in use.

The Solution: UniRide provides a secure digital platform where registration is exclusive to university email addresses and safety is ensured via official criminal record checks.

Value Proposition:

Trust: A closed community of verified students.

Accessibility: Vehicles located directly within or near the campus area

Affordability: Peer-to-peer sharing offers lower rates than market conditions.

Security: Features an escrow payment system and digital rental agreements to protect both parties.

Project Scope

User registration and profile management via university email.

Verification of criminal records and driver's licenses.

Car listing management, including pricing and availability calendars.

Secure payment infrastructure with an escrow system.

Mutual rating and review system with damage reporting.

Stakeholders

Car Owners: Students looking to earn additional income

Renters: Students seeking affordable and accessible vehicles

System Administrators: The UniRide team managing the platform and payment security.

Constraints

Users must register with a valid university email address.

Renters' credit card limits must meet specific vehicle segment requirements.

Completion of the criminal record verification process

Personas

Persona 1 – Çağrı Kaya

- **Age:** 23
- **University:** Yaşar University
- **Situation:** Owns a car but does not use it 3 days a week
- **Goal:** Earn additional income by renting out his car

Persona 2 – Yaren Toylak

- **Age:** 20
- **University:** Ege University
- **Situation:** Does not own a car and needs affordable transportation to visit her family
- **Goal:** Find a safe, affordable, and easily accessible car near campus

Scenario for Persona 2 (Renter – Yaren)

Yaren wants to rent a car to visit her family for the weekend. She registers on UniRide using her university email and completes identity and driver's license verification. After logging in, she searches for available vehicles by selecting date and price filters. She finds a suitable car listed by Çağrı and sends a reservation request.

Çağrı reviews Yaren's profile and rating, then approves the request. The system securely processes the payment and generates a digital agreement. Yaren uses the car and returns it on time. After the rental, both users rate each other.

Scenario for Persona 1 (Car Owner – Çağrı)

Çağrı wants to earn extra income from his unused car. He registers and verifies his account, then adds his vehicle with pricing and availability details. When he receives a reservation request, he reviews the renter's profile before approving it. After the rental is completed, he checks the car for damage and receives the payment through the system.

1.2 User Stories & Prioritized Feature List

High Priority

ID	User Story	Feature
US01	As a student , I want to register using my university email so that I can access a community exclusive to verified students.	Identity Service: University Email Authentication
US02	As a user , I want to complete my criminal record verification so that I can safely participate in the rental ecosystem.	Identity Service: Criminal Record Verification
US03	As a car owner , I want to list my vehicle with price and availability so that potential renters can find my car.	Vehicle Service: Car Listing & Availability
US04	As a renter , I want to search and filter cars by date and price so that I can find a vehicle within my budget.	Vehicle Service: Search & Filtering
US05	As a user , I want a secure escrow payment system so that my financial transactions are protected until the rental is completed.	Payment Service: Secure Escrow System

Medium Priority

ID	User Story	Feature
US06	As a car owner , I want to review the renter's profile and rating so that I can make a secure decision before approving a request.	Reservation Service: Profile Review & Approval

US07	As a renter , I want to send a reservation request so that I can initiate the booking process.	Reservation Service: Booking Request
US08	As a user , I want to track the rental status (Requested, Approved, In Use, Returned) so that the entire process is transparent.	Reservation Service: Real-time Status Tracking

Low Priority

ID	User Story	Feature
US09	As a user , I want to rate and review the other party so that we can maintain a high-trust environment.	Review Service: Mutual Rating & Review
US10	As a car owner , I want the payment to be released only after a successful damage inspection so that my income is guaranteed.	Review Service: Damage Reporting & Payment Release

1.3 Architecture Diagrams

Architecture Style: Microservices

UniRide is built using a microservices architecture. This architecture was chosen because the system has different parts, and each part has its own job, data, and processes. These parts include user verification, vehicle management, booking, payment, review, messaging, and damage management. With microservices, each part can be developed, updated, and scaled independently.

This architecture also supports the main quality goals of the system. For scalability, the platform should handle more users and reservations in the future, and it can grow to support multiple campuses or cities. For security, important processes like identity verification and payment are separated into different services, which makes them safer. For reliability, dividing the system into smaller parts helps prevent one problem from affecting the whole system. The diagram below shows the overall structure of the system and how different services interact with each other.

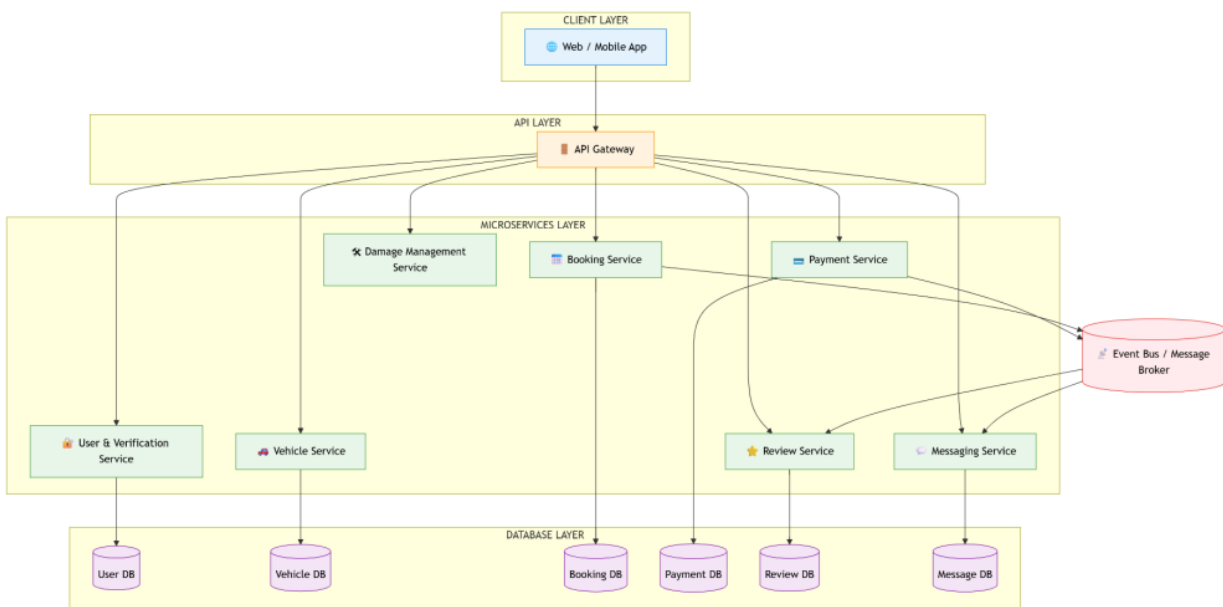


Figure 1: High level diagram for micro services

1.4 UML Diagrams

Use Case Diagram:

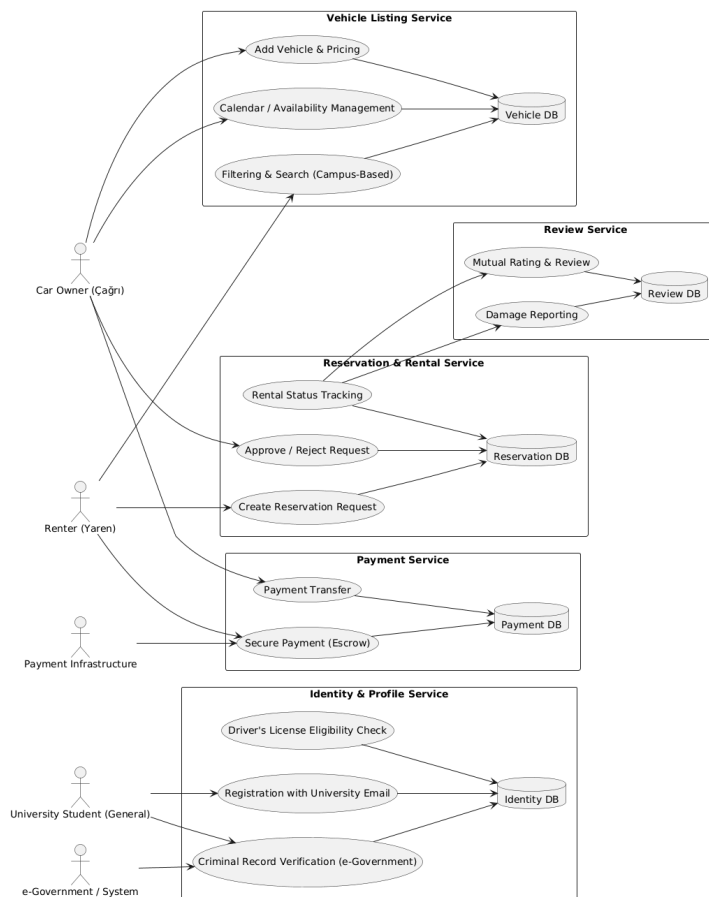


Figure 2: Use case diagram

The use case diagram shows the main actors and their interactions with the system. It includes car owners and renters, and key actions such as registration, reservation, payment, and review.

The class diagram:

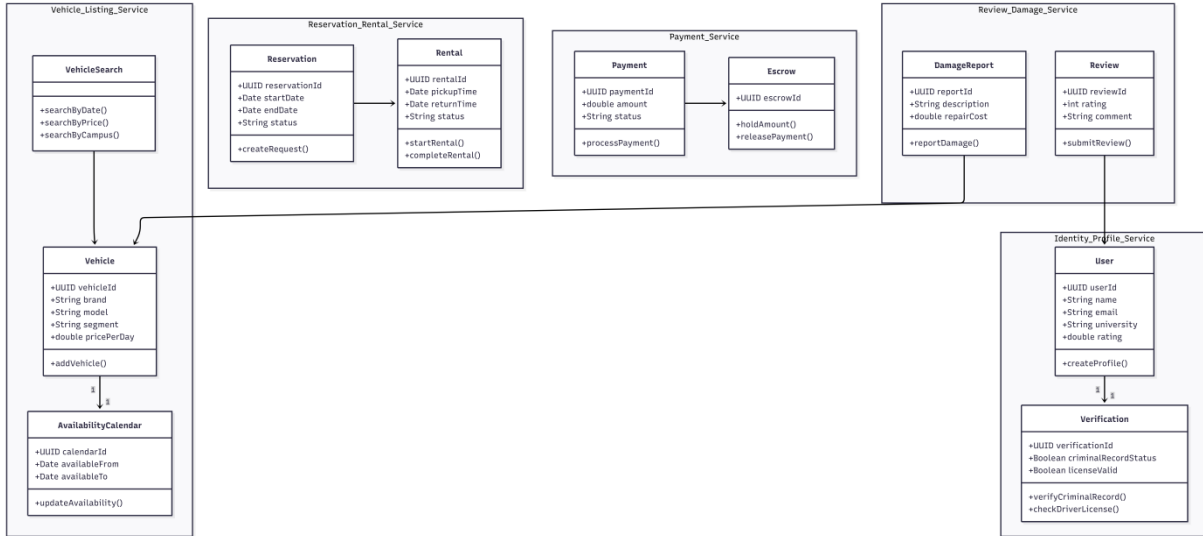


Figure 3 : The class diagram

The class diagram presents the main classes of the system, including their attributes and methods. It shows the relationships between important entities such as user, vehicle, reservation, and payment.

The sequence diagram:

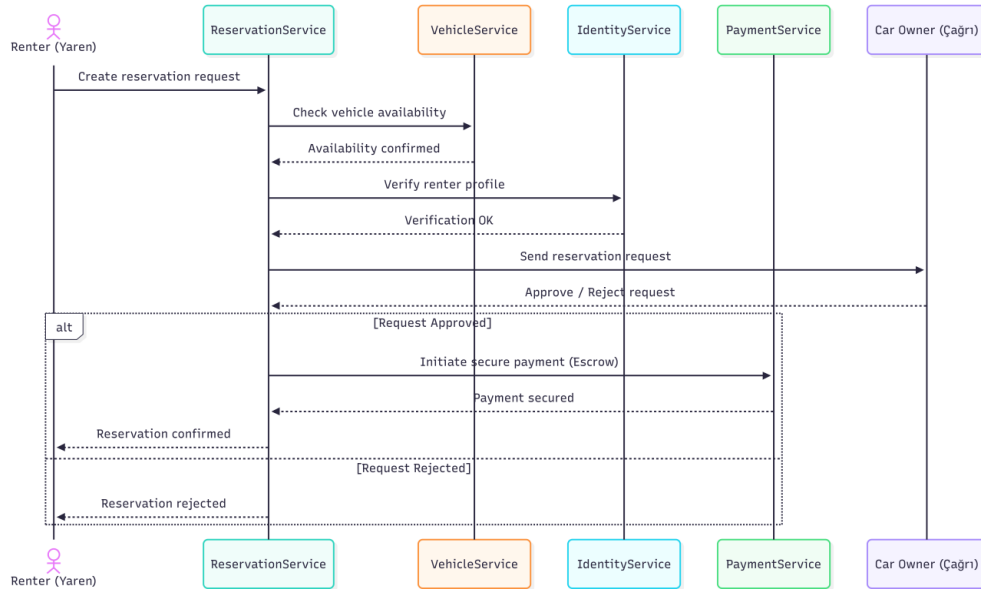


Figure 4: The sequence diagram

The sequence diagram explains the flow of the reservation process. It shows how different services interact step by step, from creating a request to completing the payment.

1.5 Selected Design Pattern

Pattern Name: Facade Pattern

What is Facade Pattern?

The Facade Pattern is a structural design pattern that provides a simplified interface to a complex system of multiple subsystems. Instead of making the client communicate with many different services, it offers one central interface that manages the process.

Why we chose this pattern?

In the UniRide system, renting a car requires interaction with several subsystems such as user verification, car availability checking, booking management, and payment processing. If the client communicates with all these services directly, the system becomes more complex and harder to maintain. Therefore, the Facade Pattern was chosen to simplify this process.

How it is used in our Project?

In our Project the RideFacade class acts as a single entry point for the car rental process. When the user wants to rent a car, the client only calls the `rent_car()` method. Then the Facade internally coordinates user verification, car availability checking, booking creation, and payment processing. This pattern improves system maintainability and reduces coupling between components.

Scenario: When a student wants to rent a car in UniRide, the system must complete four steps: verify the user, check car availability, create a booking, and process the payment. Instead of calling each subsystem separately, the client sends one request to the RideFacade class.

Code Snippet:

```
class RideFacade:

    def __init__(self):

        self.user_service = UserService()

        self.car_service = CarService()

        self.booking_service = BookingService()

        self.payment_service = PaymentService()

    def rent_car(self, user_id, car_id, amount):

        self.user_service.verify_user(user_id)
```

```

self.car_service.check_availability(car_id)

self.booking_service.create_booking(user_id, car_id)

self.payment_service.process_payment(amount)

facade = RideFacade()

facade.rent_car("Yaren", "Toyota", 50)

```

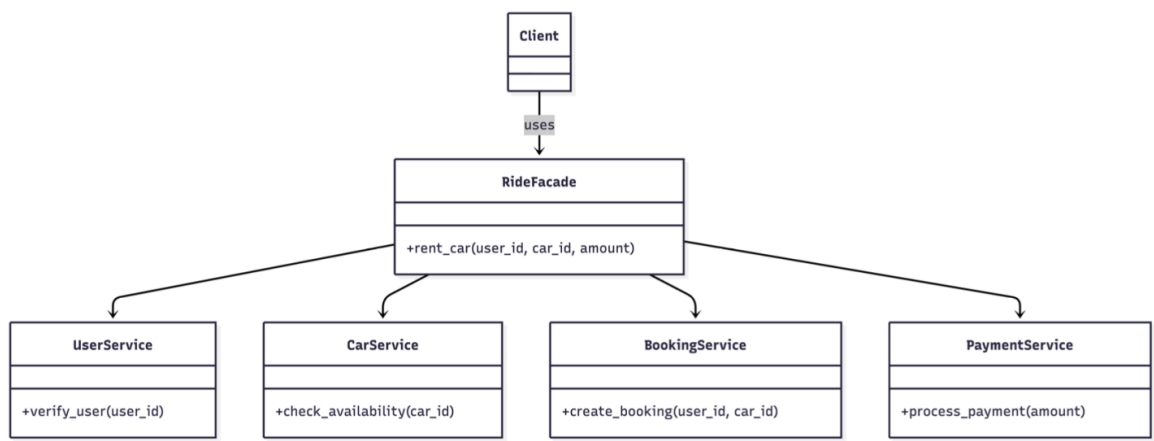


Figure 5 : Facade Pattern

Facade Pattern structure used in the UniRide system. The client interacts only with the RideFacade, while the Facade coordinates all subsystem services internally

1.6 GitHub Repository

The project is managed using GitHub for version control and team collaboration.

Repository URL: <https://github.com/nisakocagenis/uniride-teamwork>

The repository includes a structured README file describing the project and its features. GitHub was also used to track project progress and manage team contributions.

The figure 6 shows the GitHub repository including the README file, with some of the commits visible in the commit history.

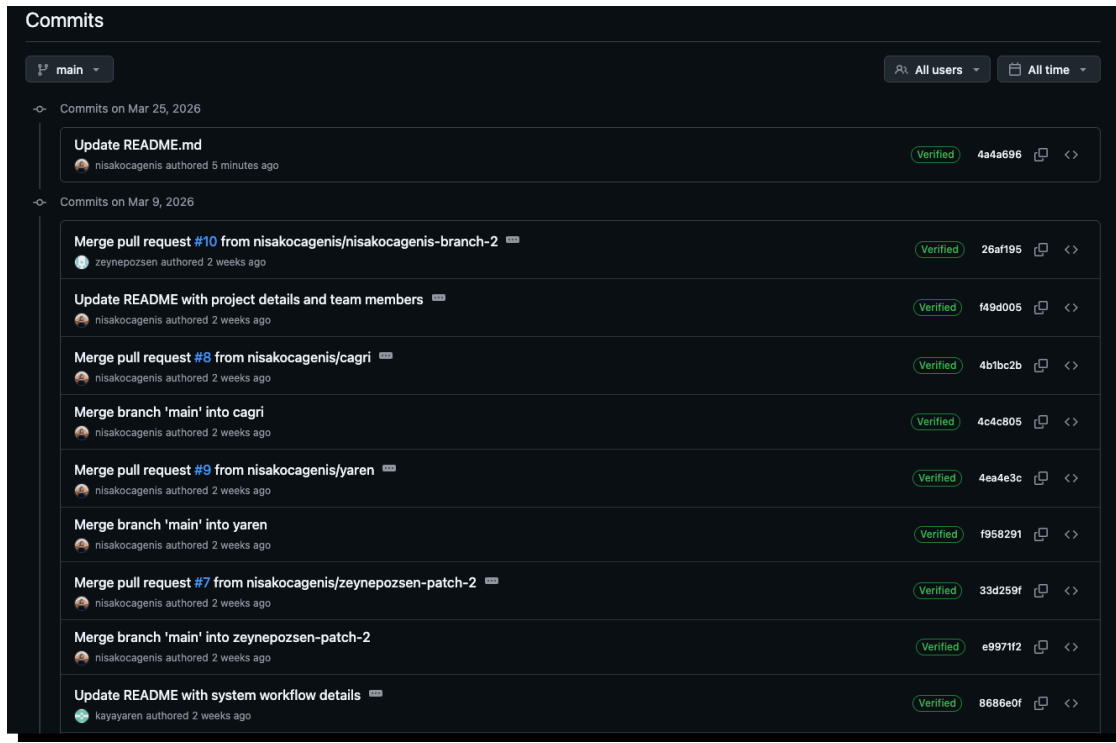


Figure 6: Repository commits history

1.7 Jira Project Management

Jira was used to manage project tasks, track development progress, and organize the workflow using Agile methodology. The product backlog contains prioritized user stories, and sprint planning was used to assign tasks and manage development iterations.

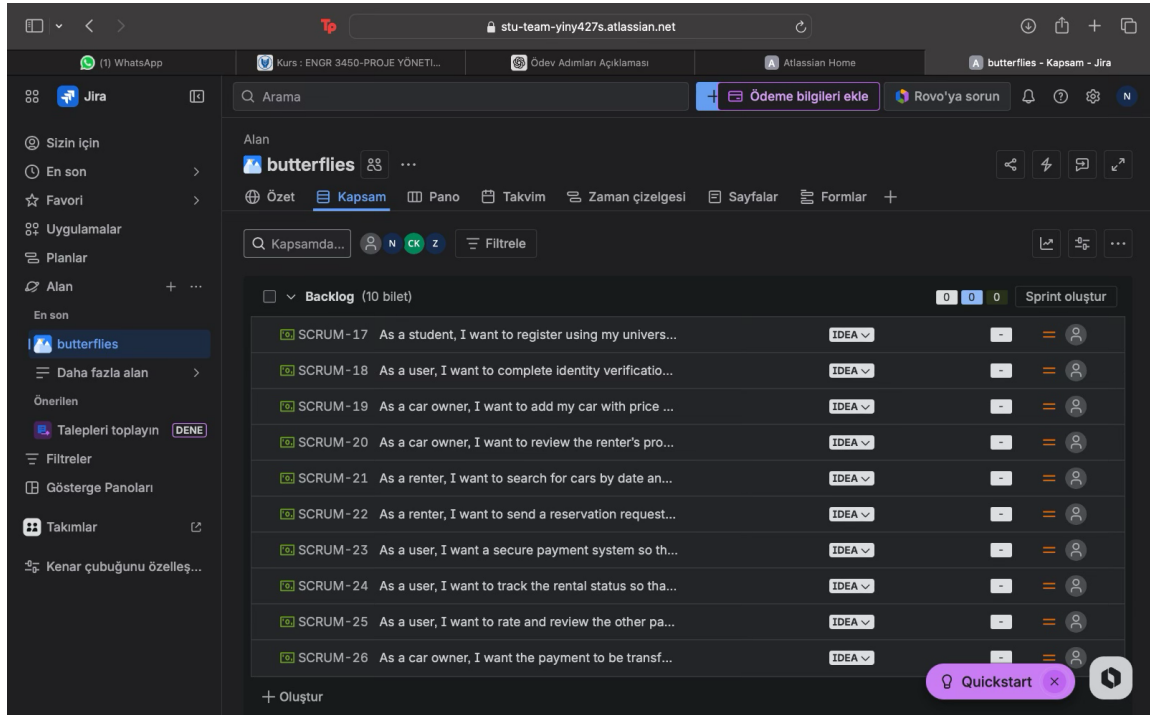


Figure 7: Product Backlog in Jira

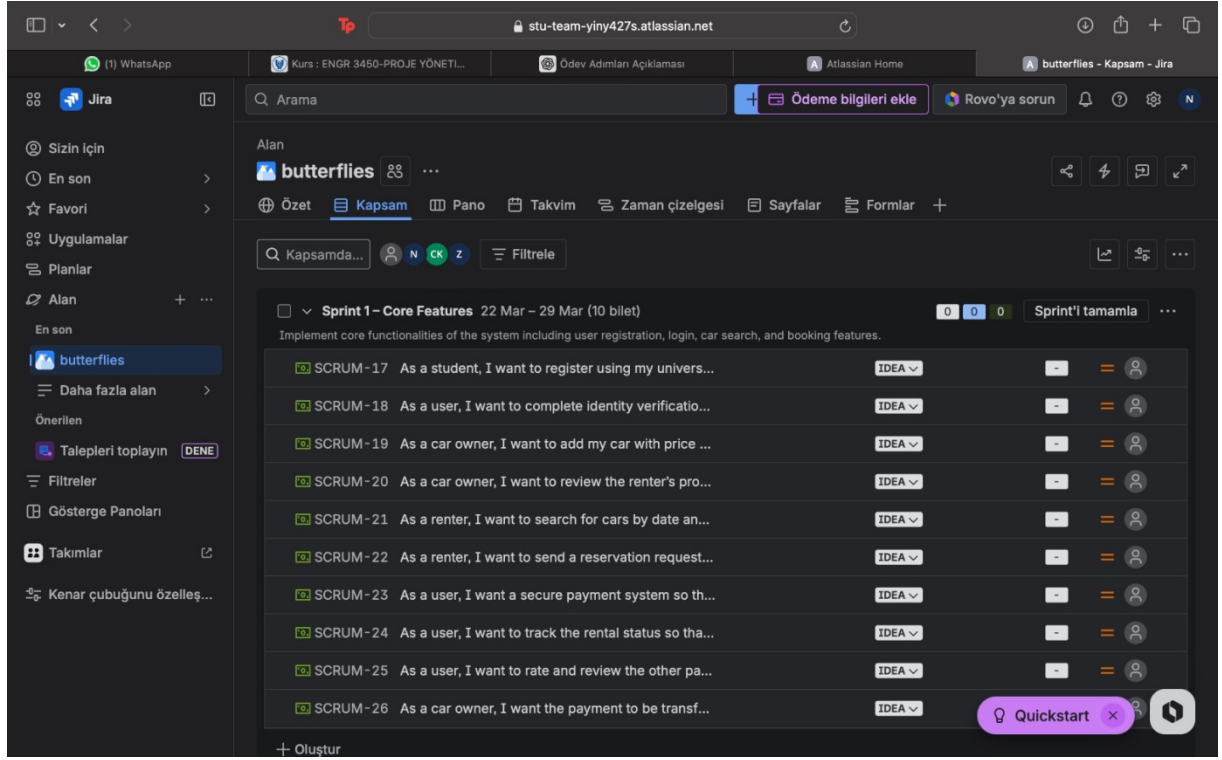


Figure 8: Sprint Planning in Jira

The figure 7 shows the prioritized all user stories in the product backlog Product Backlog in Jira

The figure 8 shows an active sprint with assigned user stories, demonstrating how tasks are organized and managed during the development process.

Student information:

Nisa KOCAGENİŞ - 22070001037

Çağrı KAYA - 22070001019

Zeynep ÖZŞEN - 23070001024

Yaren KAYA - 23070001086